

Roteiro narrado — BioByte Sentinela v5

Projeto-teste: BioByte Sentinela v5 · vigilância de infecção de corrente sanguínea associada a cateter

Pasta das telas: docs/apresentacao/demo-biobyte-v5/telas/

Como ler: cada cena traz **a tela** (caminho do arquivo), **a narração** (texto exato, no tom do narrador) e **a produção** (o que destacar, quanto tempo, o que evitar).

Duração estimada: 11 a 13 minutos.

Regra deste roteiro: nada é afirmado que não esteja na tela. Onde o sistema falhou, a cena mostra a falha.

Bloco 1 — O problema que a versão 5 veio resolver

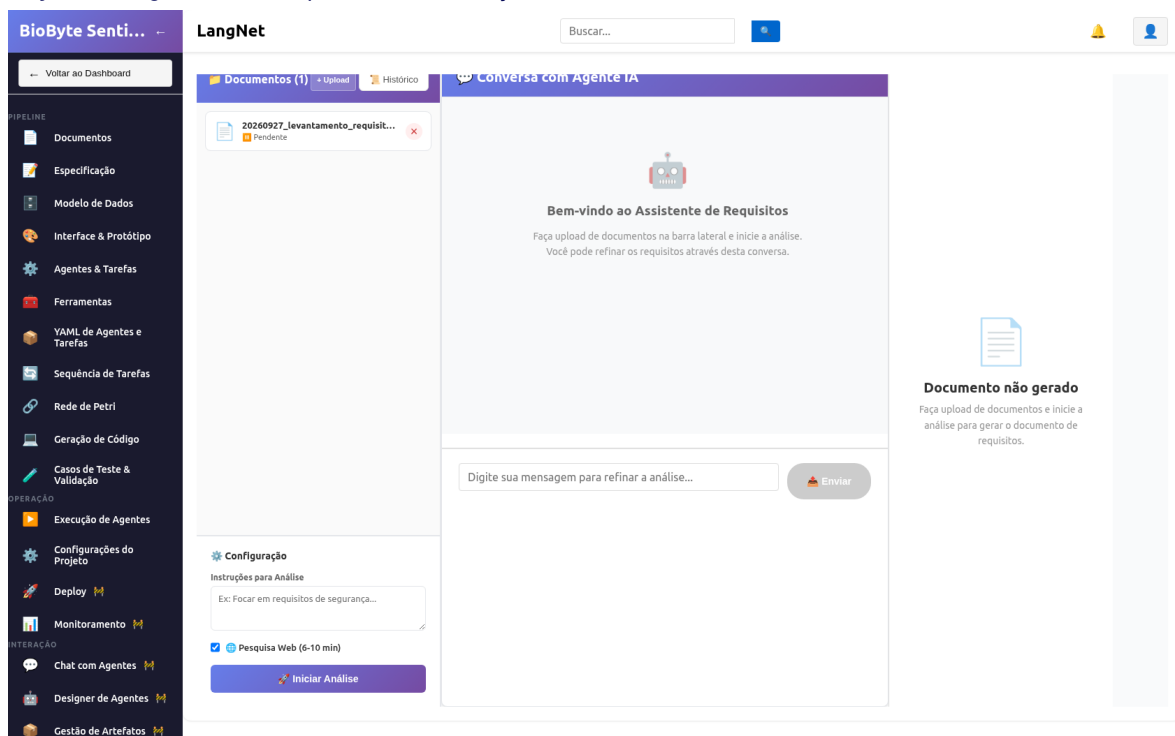
Cena 1 · O pedido do hospital

Tela: telas/01-documento-fonte.png

Narração:

Isto é o levantamento de requisitos do BioByte Sentinela. Uma médica infectologista, uma enfermeira de controle de infecção e um analista de laboratório contaram o que precisam. O problema deles é tempo: entre a hemocultura ficar pronta no laboratório e alguém perceber que aquele paciente tem uma infecção, às vezes passam dois dias. E quando o germe é multirresistente, dois dias é muita coisa.

Produção: 12 segundos. Destaque o bloco de citação da médica. Não leia o documento inteiro.



telas/01-documento-fonte.png

Cena 2 · A decisão que mudou tudo: o que é inteligência artificial e o que não é

Tela: telas/02b-requisitos-coluna-natureza.png

Narração:

Este é o documento de requisitos que o sistema produziu a partir daquela conversa. Repare na marcação de cada requisito. Um sistema hospitalar tem duas metades bem diferentes. Uma é cadastro, consulta, relatório — trabalho de sempre, que um programa comum resolve. A outra é julgamento clínico: ler um antibiograma e decidir, escrever o alerta que a equipe vai ler. Só essa segunda metade é trabalho de agente.

Na versão anterior essa distinção não existia em lugar nenhum, e o sistema tratou quarenta e cinco coisas como trabalho de agente — inclusive cadastro de usuário. Agora a separação está declarada requisito por requisito, e vem do próprio documento do hospital.

Produção: 25 segundos. Destaque com moldura a coluna de natureza, mostrando os dois valores lado a lado. Esta é a cena mais importante do primeiro bloco — dê tempo de ler.

BioByte Senti...LangNet

Buscar...

Voltar ao Dashboard

PIPELINE

Documentos

Especificação

Modelo de Dados

Interface & Protótipo

Agentes & Tarefas

Ferramentas

YAML de Agentes e Tarefas

Sequência de Tarefas

Rede de Petri

Geração de Código

Casos de Teste & Validação

OPERAÇÃO

Execução de Agentes

Configurações do Projeto

Deploy

Monitoramento

INTERAÇÃO

Chat com Agentes

Designer de Agentes

Gestão de Artefatos

FR-021	RED	convencional	Tratar mensagem de erro do laboratório	O sistema deve tratar o retorno do laboratório com mensagem explicando problema de cadastro da amostra, sem quebrar e sem inventar resultado.	high	BioByte	Consultar laboratório por número do paciente
FR-022	RED	convencional	Evitar duplicação de resultado importado	O sistema deve reconhecer quando o mesmo resultado já foi buscado e não criar registro repetido.	high	BioByte	Consultar laboratório por número do paciente
FR-024	RED	agêntica	Aplicar critério NHSN vigente	O sistema deve aplicar o critério NHSN vigente ao caso, com base na microbiologia, para classificá-lo.	high	sistema; médico; coordenadora CCIH	—
FR-025	RED	agêntica	Classificar caso como confirmada descartada pendente	O sistema deve decidir entre as classificações confirmada, descartada ou pendente para o caso.	high	sistema; médico	Aplicar critério NHSN vigente
FR-026	RED	convencional	Registrar critério e versão aplicados	O sistema deve registrar, junto à classificação, qual critério foi aplicado e qual versão dele sustentou a decisão.	high	sistema; coordenadora CCIH; auditor	Aplicar critério NHSN vigente
FR-027	RED	convencional	Cadastrar critérios com	O sistema deve permitir cadastrar critérios contendo versão nome	high	coordenadora CCIH	—

Table of Contents

Auto-generated from document headings

Document Stats

LINES: 1159

CHARACTERS: 172.379

SECTIONS: 22

telas/02b-requisitos-coluna-natureza.png

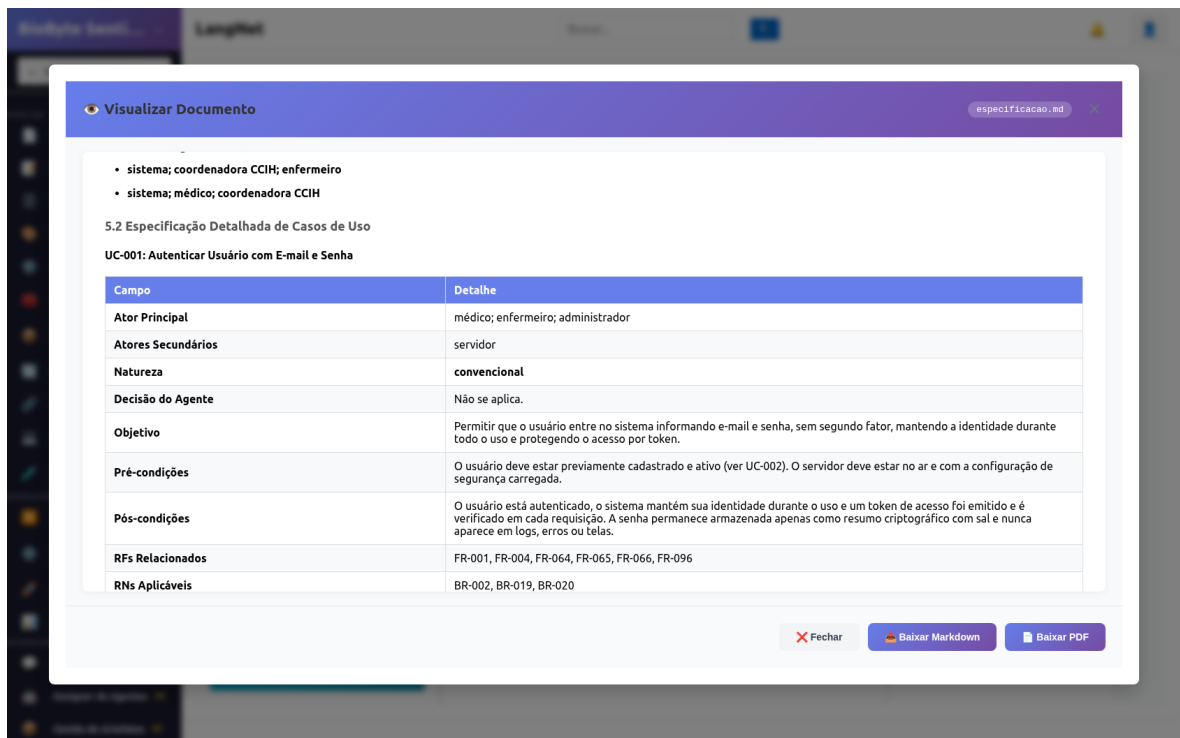
Cena 3 · A separação chega ao caso de uso

Tela: telas/03b-caso-de-uso-natureza.png

Narração:

A mesma separação desce para o caso de uso. Cada caso de uso agora declara a sua natureza e, quando é de agente, declara também o que exatamente o agente decide. Não é uma anotação decorativa: é o que o resto da linha de produção vai ler para saber quem faz o quê.

Produção: 15 segundos. Destaque as duas linhas — natureza e decisão do agente.



telas/03b-caso-de-uso-natureza.png

Bloco 2 — A linha de produção

Cena 4 · Cinco tarefas, dois agentes

Tela: telas/04b-cinco-tarefas-dois-agentes.png

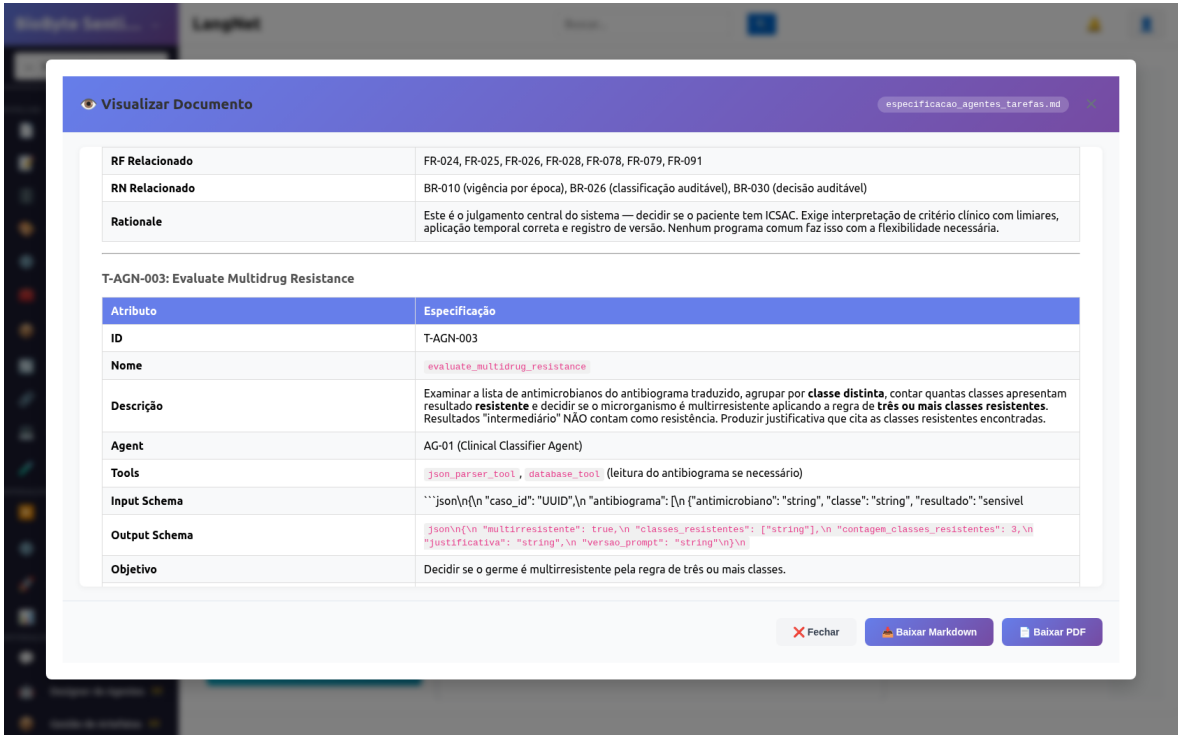
Narração:

E aqui está o resultado. Trinta e um casos de uso entraram; seis são de agente; e a especificação de agentes e tarefas produziu cinco tarefas distribuídas em dois agentes. Traduzir o resultado do laboratório para o vocabulário do hospital. Classificar o caso pelo critério da norma. Avaliar multirresistência. Recomendar o pacote de medidas com justificativa. Redigir o alerta para a equipe.

>

Cinco. Na versão anterior eram quarenta e cinco.

Produção: 22 segundos. Destaque a lista das cinco tarefas. Se couber, mostre o número "45" da versão anterior num canto, para contraste.



telas/04b-cinco-tarefas-dois-agentes.png

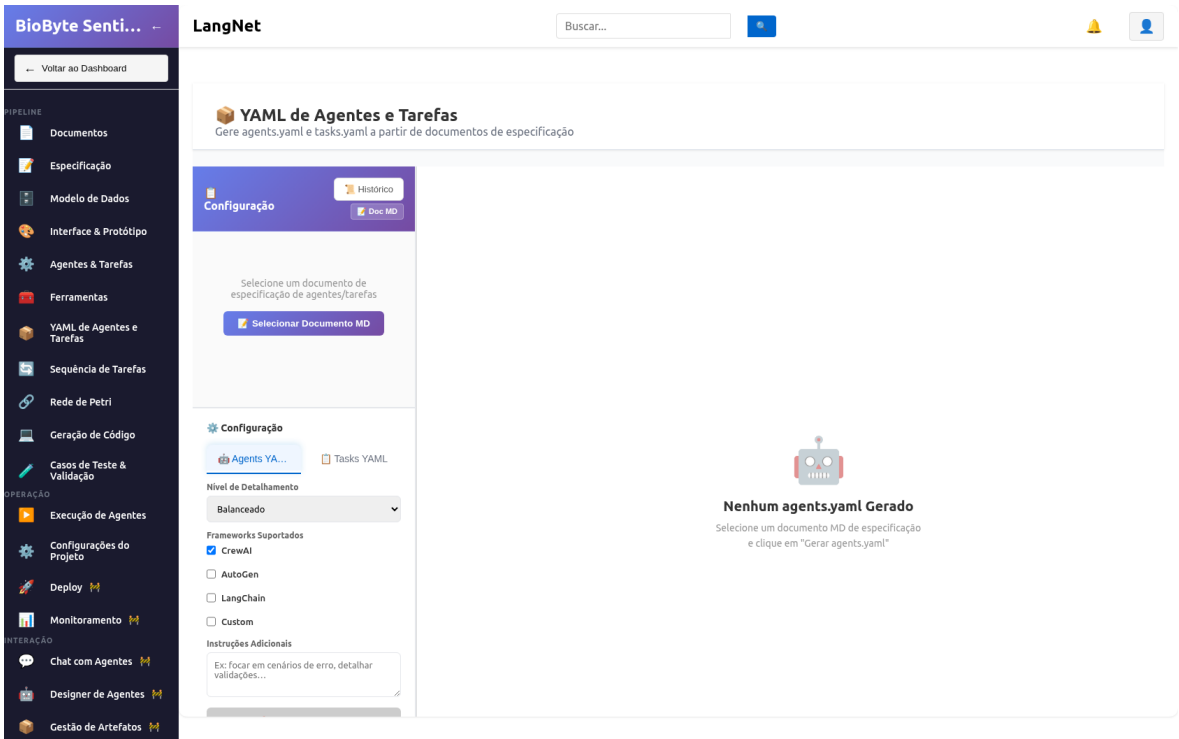
Cena 5 · O contrato com o CrewAI

Tela: telas/05-yaml-crewai.png

Narração:

As tarefas viram dois arquivos de configuração no formato que o CrewAI entende. Um descreve os agentes — papel, objetivo, história. O outro descreve as tarefas — o que fazer e o que se espera de saída. Nada além disso: quem executa cada tarefa e com quais ferramentas é decidido na geração do código, não aqui.

Produção: 14 segundos. Destaque description e expected_output numa tarefa.



Cena 5b · Quem implementa cada ferramenta

Tela: telas/61-ferramentas-resultado.png

Narração:

Antes de gerar o sistema, uma pergunta precisa de resposta: cada ferramenta que os agentes vão usar, de onde vem? Esta etapa faz o inventário. Três ferramentas — consulta ao banco, leitura de dados estruturados e busca por semelhança — e as três com implementação declarada, nenhuma pendente.

>

Isso não é burocracia. Sem essa declaração, quem monta o sistema improvisa; e um agente montado sem ferramenta responde, educadamente, que não conseguiu obter o dado.

Produção: 20 segundos. Destaque a coluna de origem e o contador "3 com implementação · 0 pendentes".

BioByte Senti...

LangNet

Buscar...

Voltar ao Dashboard

PIPELINE

Documentos

Especificação

Modelo de Dados

Interface & Protótipo

Agentes & Tarefas

Ferramentas

YAML de Agentes e Tarefas

Sequência de Tarefas

Rede de Petri

Geração de Código

Casos de Teste & Validação

OPERAÇÃO

Execução de Agentes

Configurações do Projeto

Deploy

Monitoramento

INTERAÇÃO

Chat com Agentes

Designer de Agentes

Gestão de Artefatos

Ferramentas

v1 · 3/3 resolvidas · 27/09/2026, 23:35:18

Gerar a partir do ATS

Aprovar etapa

FERRAMENTAS3

COM IMPLEMENTAÇÃO3

PENDENTES0

VERSÃOv1

Todas as ferramentas têm implementação declarada. Etapa pronta para aprovação.

Ferramenta	Origem	Implementação	Usada por
database_tool	Biblioteca do gerador	biblioteca do gerador (database_tool)	agent:clinical_classifier_agent, agent:treatment_recommender_agent, tarefa:translate_lab_result_to_hospital_vocabulary, tarefa:classify_case_by_nhsn_criterion, tarefa:evaluate_multidrug_resistance, tarefa:draft_alert_text_for_care_team, tarefa:recommend_bundle_with_justification
json_parser_tool	Determinística	regra em 5 passo(s) — vira código na geração	agent:technical_classifier_agent, agent:treatment_recommender_agent, tarefa:translate_lab_result_to_hospital_vocabulary, tarefa:classify_case_by_nhsn_criterion, tarefa:evaluate_multidrug_resistance, tarefa:draft_alert_text_for_care_team, tarefa:recommend_bundle_with_justification
vector_search_tool	Biblioteca do gerador	biblioteca do gerador (vector_search_tool)	agent:technical_classifier_agent, tarefa:translate_lab_result_to_hospital_vocabulary, tarefa:classify_case_by_nhsn_criterion

Selecione uma ferramenta na tabela para ver e editar o contrato.

Refinar com o agente

Ex.: a validação de senha é determinística; a regra é conferir o hash

Enviar

Versões

v1geração inicial a partir do ATS

telas/61-ferramentas-resultado.png

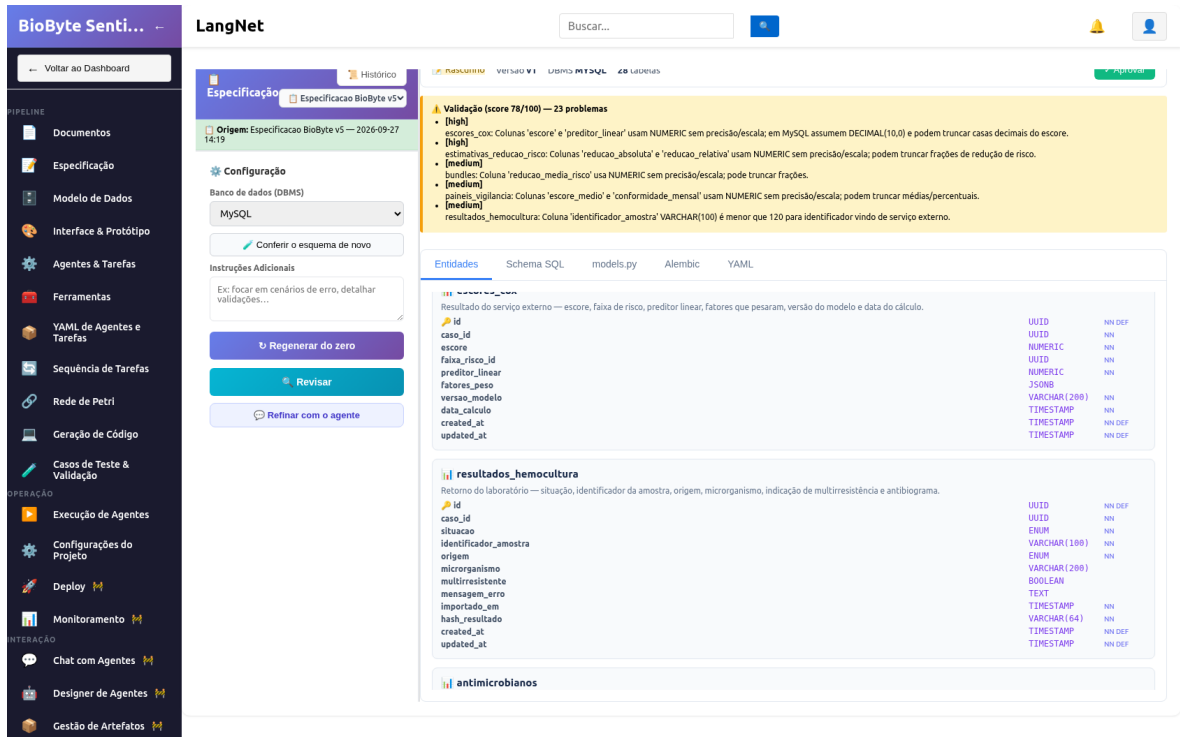
Cena 6 · O modelo de dados

Tela: telas/06b-modelo-antibiograma.png

Narração:

O modelo de dados sai da mesma fonte. Vinte e oito tabelas, entre elas as que vão sustentar a decisão clínica: o resultado da hemocultura, o antibiograma, os antimicrobianos com as suas classes. É contra estas tabelas que os agentes vão trabalhar — não contra um exemplo inventado.

Produção: 14 segundos. Destaque a tabela de antibiograma.



telas/06b-modelo-antibiograma.png

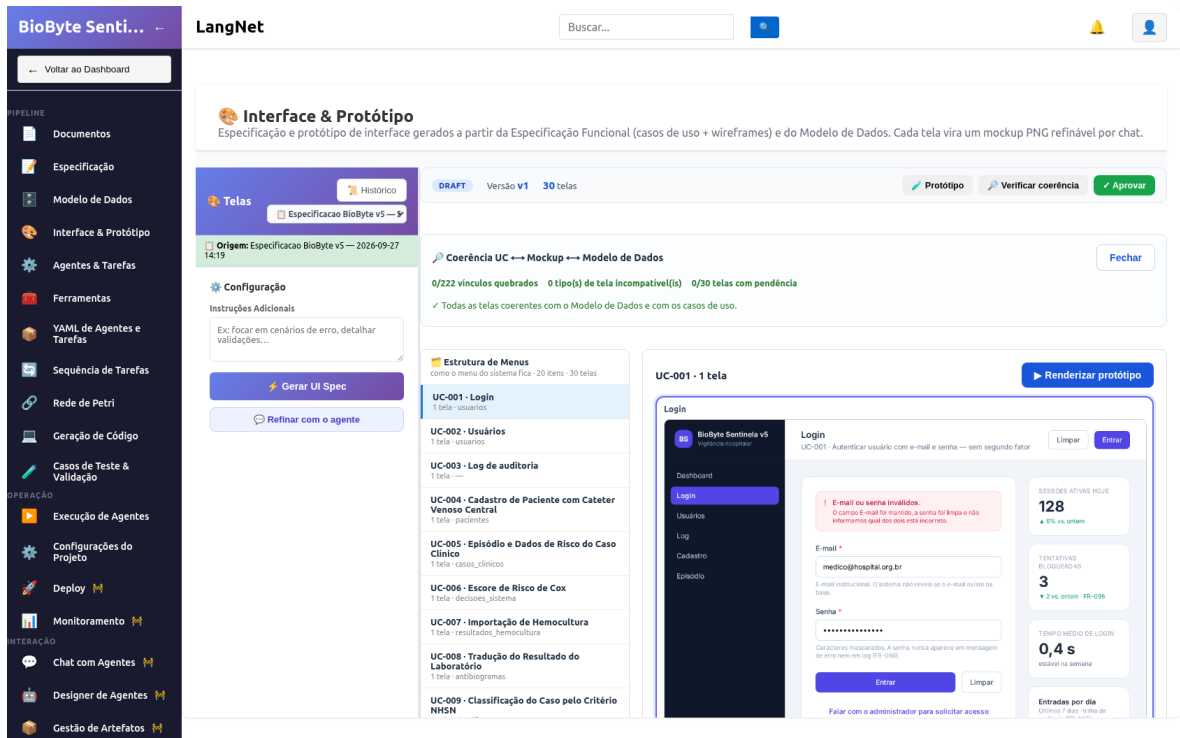
Cena 7 · As telas do hospital

Tela: telas/07-interface-telas.png

Narração:

A especificação de interface desenha as telas do sistema a partir dos casos de uso. Trinta telas. E aqui aparece de novo a separação: as telas de cadastro e relatório são convencionais; as que chamam um agente ficam marcadas como tal.

Produção: 14 segundos.



telas/07-interface-telas.png

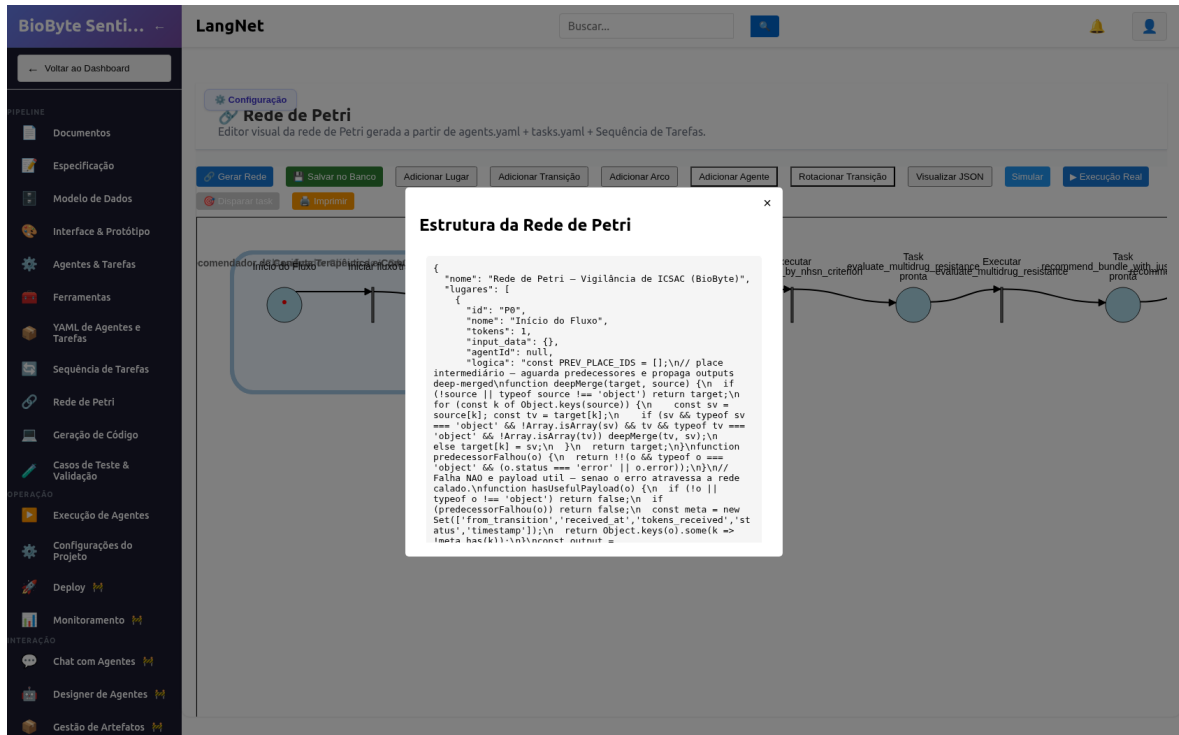
Cena 8 · A planta do fluxo

Tela: telas/08-rede-de-petri.png

Narração:

Esta é a planta do fluxo, em rede de Petri. Sete lugares, seis transições. Cada lugar é uma tarefa de agente esperando a sua vez; cada transição é a regra que libera a próxima. A marca começa no início e só avança quando o lugar anterior concluiu de verdade. Isto não é um desenho ilustrativo: é o que vai ser executado.

Produção: 18 segundos. Destaque a sequência dos cinco lugares centrais.



telas/08-rede-de-petri.png

Cena 9 · O sistema sai pronto

Tela: telas/09-codigo-gerado.png

Narração:

A geração de código monta o sistema inteiro: o servidor dos agentes, a interface do hospital, o banco com o seu esquema, e a ligação entre eles. Cento e trinta e seis arquivos.

Produção: 12 segundos.

BioByte Senti...

LangNet

Buscar...

Voltar ao Dashboard

PIPELINE

Documentos

Especificação

Modelo de Dados

Interface & Protótipo

Agentes & Tarefas

Ferramentas

YAML de Agentes e Tarefas

Sequência de Tarefas

Rede de Petri

Geração de Código

Casos de Teste & Validação

OPERAÇÃO

Execução de Agentes

Configurações do Projeto

Deploy

Monitoramento

INTERAÇÃO

Chat com Agentes

Designer de Agentes

Gestão de Artefatos

Geração de Código Python

PRONTO

Gere e gerencie o código Python da sua aplicação baseada em agentes

Download

Deploy

Executar

Gerar/Atualizar

Status da Aplicação

8.5/10

QUALIDADE

82.5%

COBERTURA

15

ARQUIVOS

1215

BUILD

Sistema de Suporte ao Cliente

Visualizar

Configurar

main.py

main.py 40 linhas 968 caracteres

Formatar

Arquivos

Build

Testes (25)

Deploy

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

#!/usr/bin/env python3

"""

Customer Support System - Main Application

Generated by LangNet Framework

"""

from customer_support_net import CustomerSupportNet

from langfuse import Langfuse

import logging

def main():

"""Initialize and run the Customer Support System"""

Configure logging

logging.basicConfig(level=logging.INFO)

logger = logging.getLogger(__name__)

Initialize Langfuse monitoring

langfuse = Langfuse()

Create and configure the Petri Net

customer_net = CustomerSupportNet()

Console de Execução

Limpar

Executar

[14:32:00] Starting customer support system

[14:32:02] Network initialized with 5 places and 4 transitions

Estado da Rede de Petri

start

0

process_query

1

police_senti...

telas/09-codigo-gerado.png

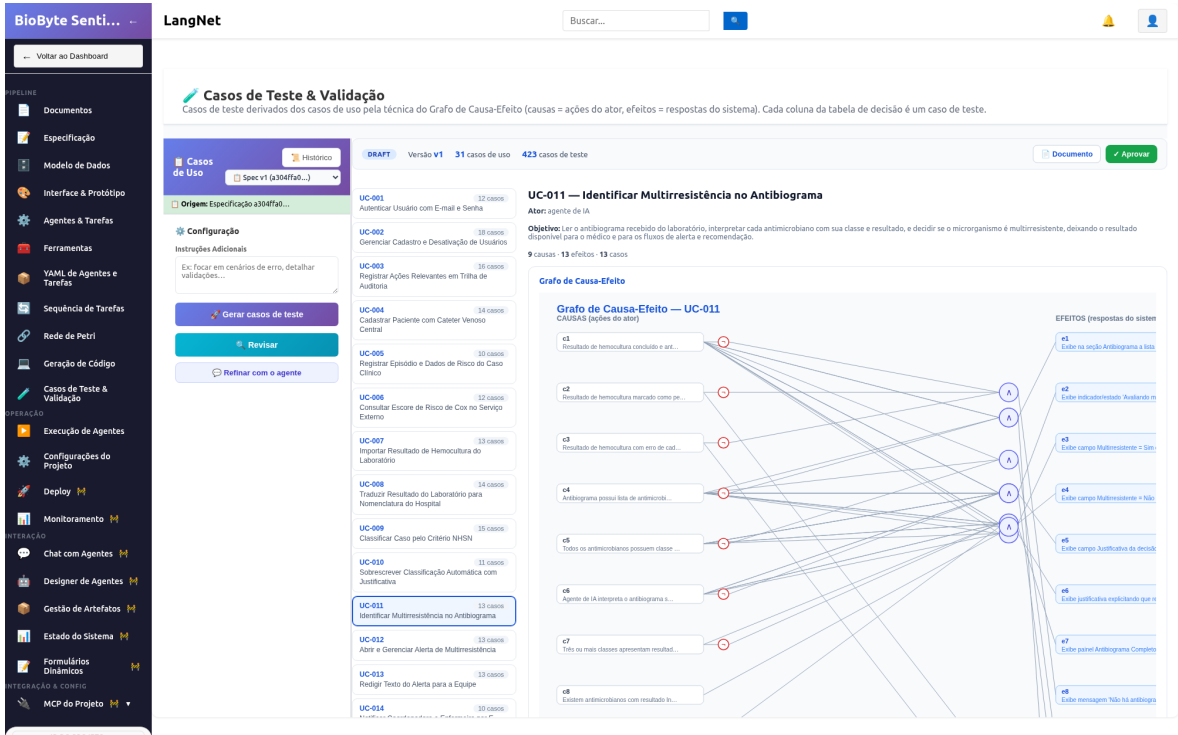
Cena 9b · De onde saem os casos de teste

Tela: telas/64-grafo-causa-efeito.png

Narração:

Os casos de teste não são escritos à mão. Esta etapa lê cada caso de uso e monta um grafo de causa e efeito. À esquerda, as causas: as ações do ator e as condições do mundo. À direita, os efeitos: o que o sistema responde. No meio, as combinações — este efeito só acontece se estas causas forem verdadeiras e aquela for falsa.

Produção: 22 segundos. Destaque um efeito e siga os arcos até as causas que o dispararam. Mostre um círculo de negação.



telas/64-grafo-causa-efeito.png

Cena 9c · A tabela de decisão e o caso pronto

Telas: telas/65-tabela-de-decisao.png e telas/66-casos-de-teste-texto.png

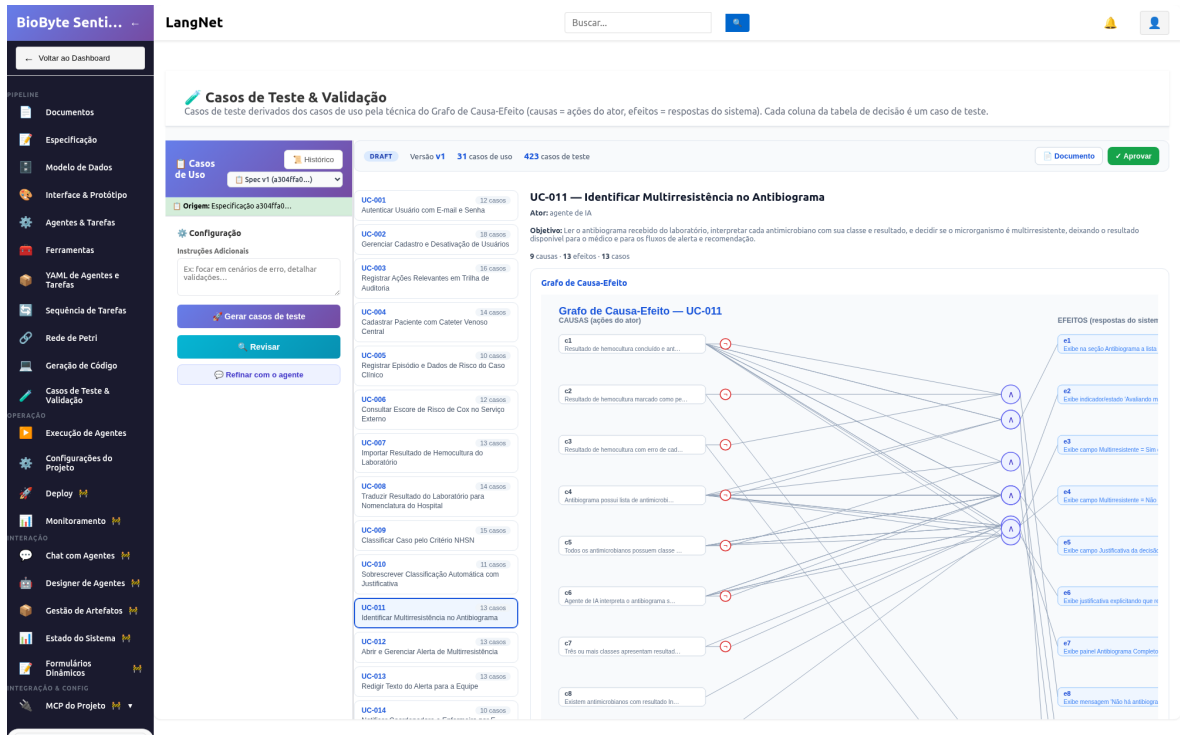
Narração:

Do grafo sai a tabela de decisão. Cada coluna é uma combinação possível — e cada coluna vira um caso de teste. Trinta e um casos de uso, quatrocentos e vinte e três casos de teste.

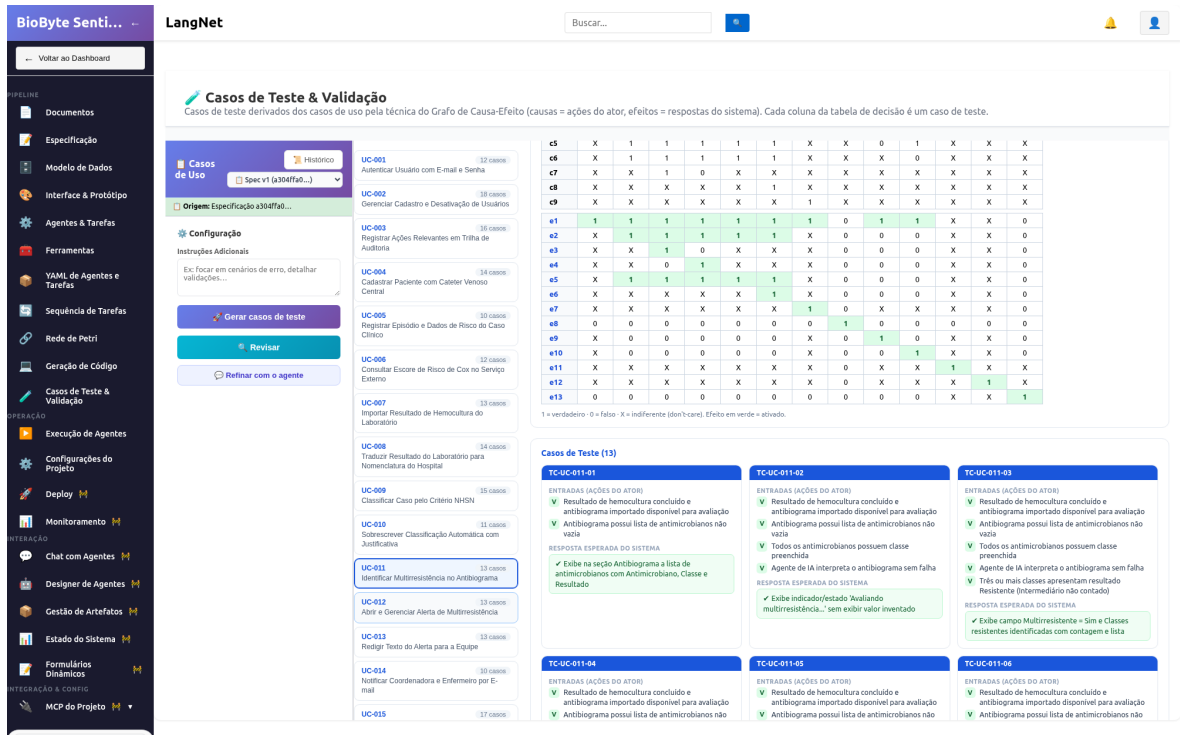
>

E cada caso vem escrito: as entradas, que são as ações do ator, e a resposta esperada do sistema. É contra isso que o sistema vai ser cobrado.

Produção: 24 segundos. Corte da tabela para o caso escrito, mostrando que a coluna virou o caso.



telas/65-tabela-de-decisao.png



telas/66-casos-de-teste-texto.png

Bloco 3 — As duas interfaces

Nota de produção: este bloco existe porque há **duas** coisas para ver, e elas são diferentes. A interface do hospital é o que o médico usa. A Bancada de Execução é o nosso instrumento — é onde se vê a rede rodando por dentro.

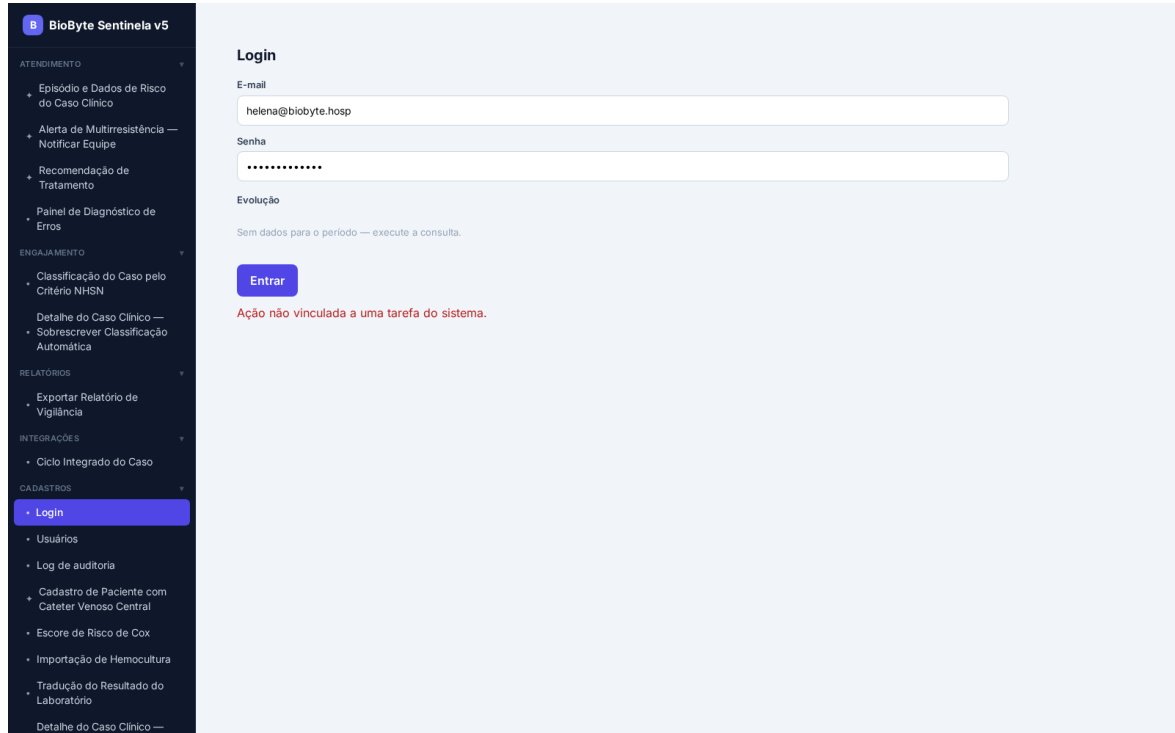
Cena 10 · A interface do hospital

Tela: telas/20-app-inicial.png

Narração:

Esta é a interface do sistema gerado — o que o hospital usa. Atendimento, engajamento, relatórios, integrações, cadastros. E repare no menu: as telas marcadas com o losango são as que chamam um agente; as outras são convencionais. A separação que começou no documento do hospital chegou até o menu lateral.

Produção: 18 segundos. Destaque dois itens do menu lado a lado — um com losango, um sem.



telas/20-app-inicial.png

Cena 10b · A tela do hospital, por dentro

Tela: telas/22-app-tela-de-agente.png

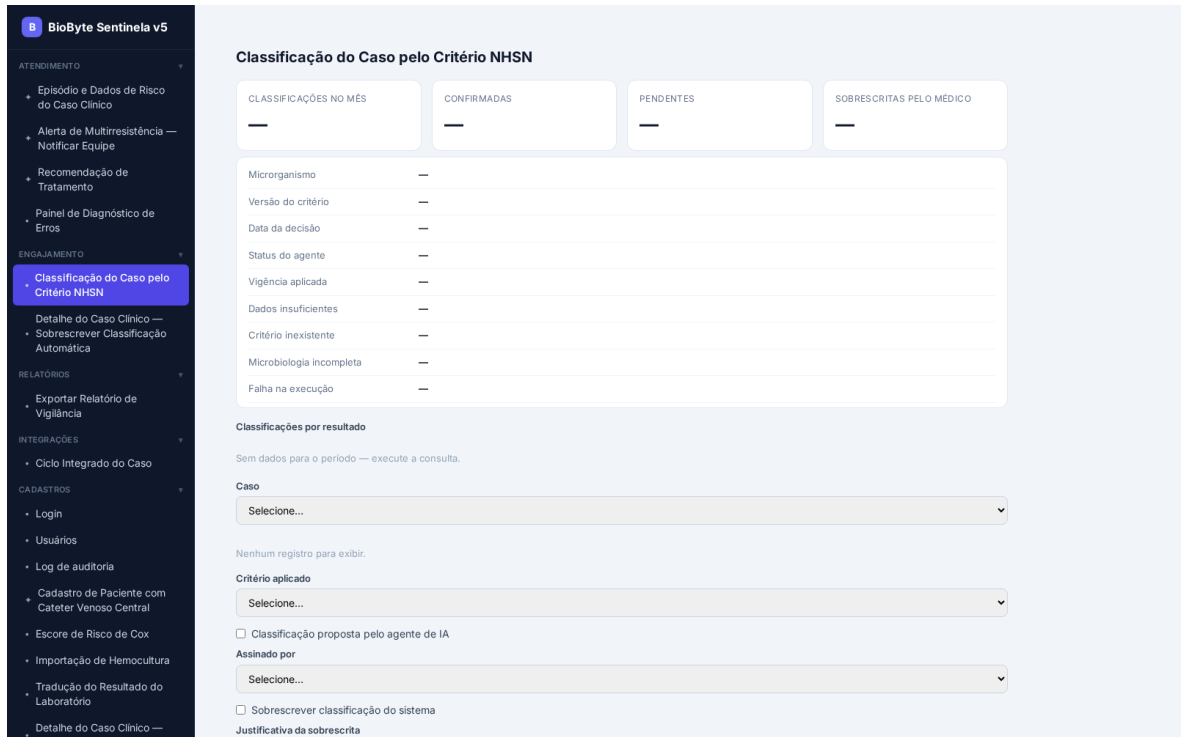
Narração:

Esta é uma tela de dentro. Classificação do caso pelo critério da norma — indicadores no topo, o resultado da decisão campo a campo, e embaixo os seletores de caso e de critério. Não é um formulário genérico: cada campo veio do caso de uso.

>

E aqui está a parte honesta. Os campos estão vazios. A tela está desenhada, mas o que a alimenta — listar os casos, carregar os critérios — ainda não tem quem execute. Ao separar o que é trabalho de agente, a versão 5 deixou de gerar tarefa para o que é convencional; e o lado convencional ainda não ganhou o seu próprio executor. É o item mais importante da lista do que falta.

Produção: 26 segundos. Destaque um indicador vazio e um seletor vazio. Não esconda: esta cena é o motivo de a versão 6 existir.



telas/22-app-tela-de-agente.png

Cena 10c · O protótipo, antes de existir sistema

Tela: telas/73-prototipo-tela.png

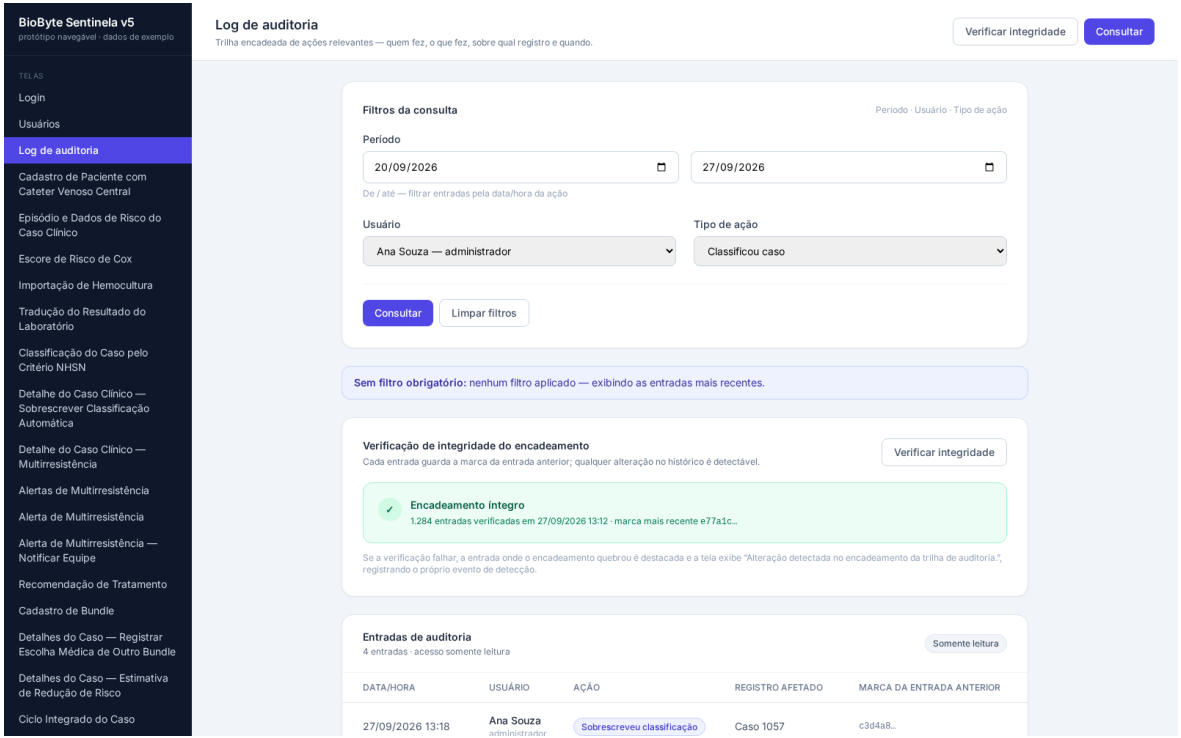
Narração:

Antes de gerar o sistema, as telas já podem ser navegadas. Isto é o protótipo: as mesmas trinta telas, ligadas pelo menu, com dados de exemplo. Aqui está a trilha de auditoria — filtros por período e por usuário, a verificação de integridade do encadeamento, e as entradas com a marca da entrada anterior.

>

É o que a tela deve fazer. Guarde esta imagem: daqui a pouco veremos a mesma tela no sistema gerado.

Produção: 22 segundos. Destaque o selo "Encadeamento íntegro · 1.284 entradas verificadas".



telas/73-prototipo-tela.png

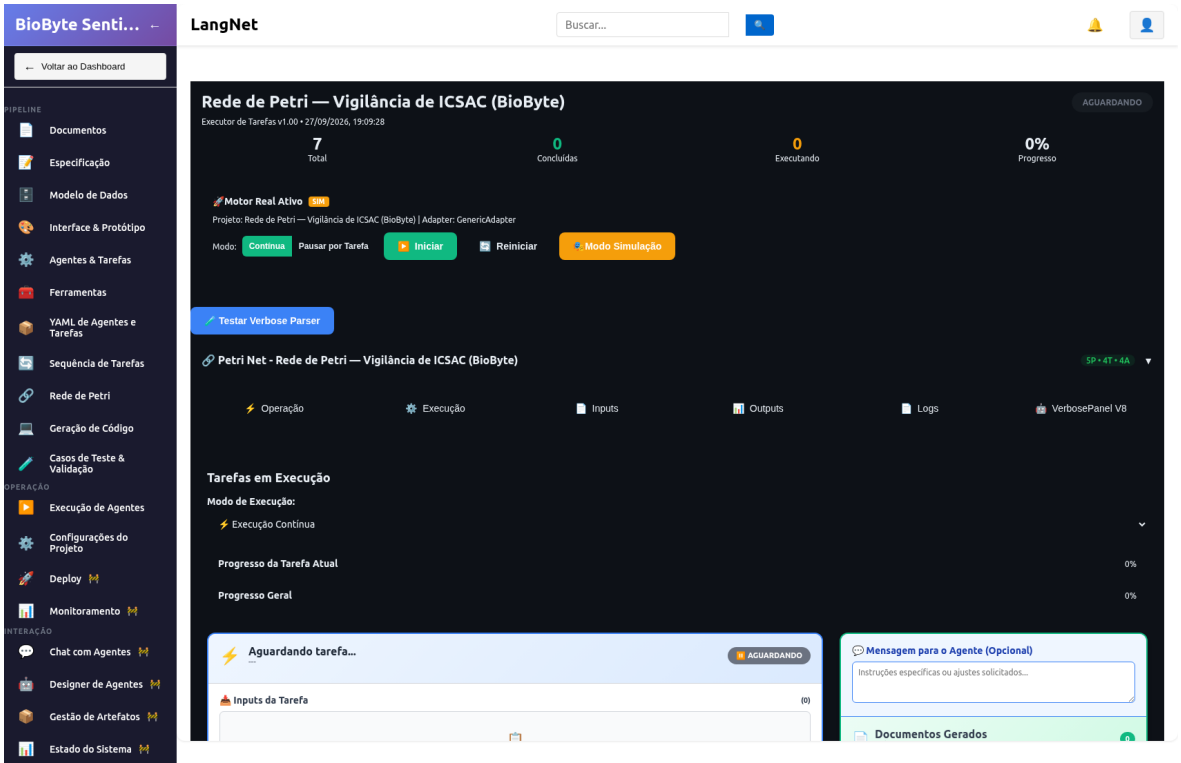
Cena 11 · A Bancada de Execução

Tela: telas/30-bancada-rede.png

Narração:

E esta é a Bancada de Execução. É a outra interface, e serve a outro propósito: aqui a rede de Petri aparece inteira e roda diante de nós. Não é uma animação — cada lugar abre uma conversa com o servidor dos agentes, manda a tarefa, e espera a resposta.

Produção: 16 segundos.



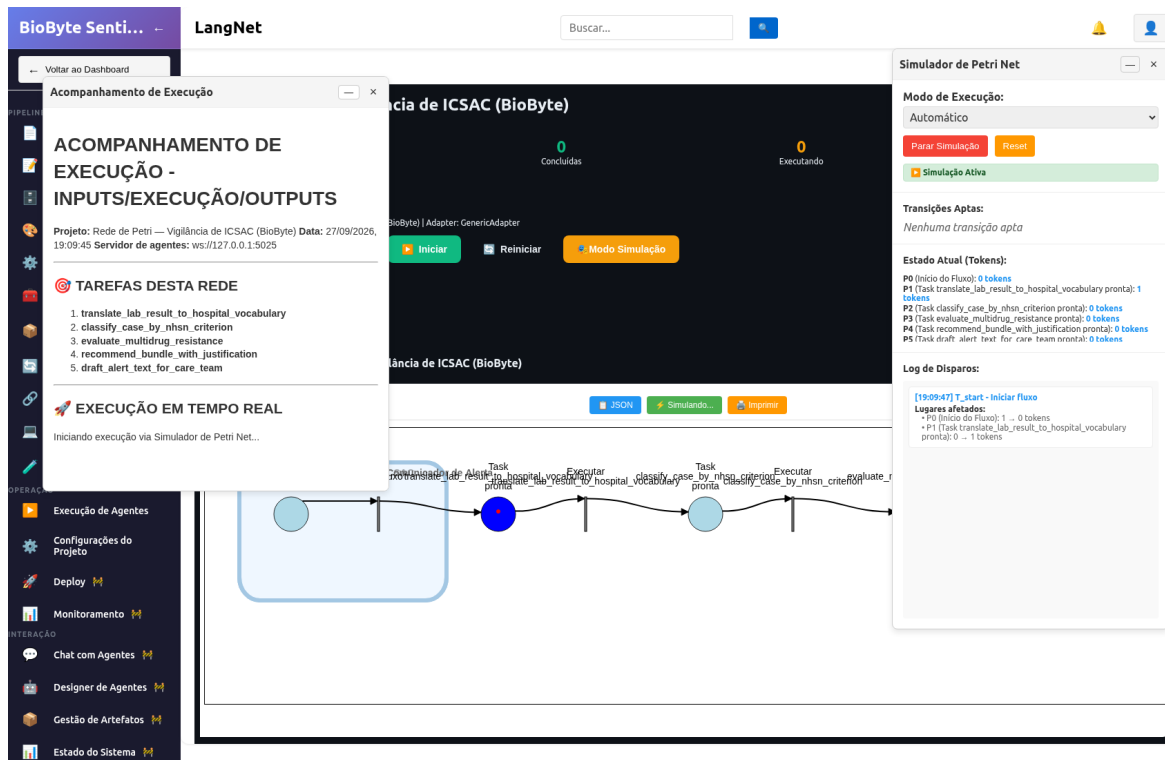
Cena 12 · A marca andando

Telas: telas/31-bancada-passo.png, telas/32-bancada-passo.png, telas/33-bancada-passo.png, telas/41-bancada-log-disparos.png

Narração:

A marca sai do início e entra na primeira tarefa: traduzir o resultado do laboratório. Concluída, a transição libera a próxima: classificar o caso. Depois, avaliar multirresistência. Cada passagem dessas é uma tarefa de verdade sendo executada, contra o banco de verdade.

Produção: 20 segundos, cortando entre as três capturas no ritmo da narração.



BioByte Senti... LangNet

Buscar...

Voltar ao Dashboard

ACOMPANHAMENTO DE EXECUÇÃO - INPUTS/EXECUÇÃO/OUTPUTS

Projeto: Rede de Petri — Vigilância de ICSAC (BioByte) Data: 27/09/2026, 19:09:45 Servidor de agentes: ws://127.0.0.1:5025

TAREFAS DESTA REDE

1. translate_lab_result_to_hospital_vocabulary
2. classify_case_by_nhsn_criterion
3. evaluate_multidrug_resistance
4. recommend_bundle_with_justification
5. draft_alert_text_for_care_team

EXECUÇÃO EM TEMPO REAL

Iniciando execução via Simulador de Petri Net...

Simulador de Petri Net

Modo de Execução: Automático

Parar Simulação Reset

Simulação Ativa

Transições Aptas: Nenhuma transição apta

Estado Atual (Tokens):

- P0 (Início do Fluxo): 0 tokens
- P1 (Task translate_lab_result_to_hospital_vocabulary pronta): 0 tokens
- P2 (Task classify_case_by_nhsn_criterion pronta): 1 tokens
- P3 (Task evaluate_multidrug_resistance pronta): 0 tokens
- P4 (Task recommend_bundle_with_justification pronta): 0 tokens
- P5 (Task draft_alert_text_for_care_team pronta): 0 tokens

Log de Disparos:

- [19:09:47] T_start - Iniciar fluxo
Lugares afetados:
 - P0 (Início do Fluxo): 1 -> 0 tokens
 - P1 (Task translate_lab_result_to_hospital_vocabulary pronta): 0 -> 1 tokens
- [19:09:53] T_translate_lab_result_to_hospital_vocabulary - Executar translate_lab_result_to_hospital_vocabulary
Lugares afetados:
 - P1 (Task translate_lab_result_to_hospital_vocabulary pronta): 1 -> 0 tokens
 - P2 (Task classify_case_by_nhsn_criterion pronta): 0 -> 1 tokens

Diagrama de Petri Net:

```
graph LR; P0((P0)) --> T1[T1: translate_lab_result_to_hospital_vocabulary]; T1 --> P1((P1)); P1 --> T2[T2: classify_case_by_nhsn_criterion]; T2 --> P2((P2)); P2 --> T3[T3: evaluate_multidrug_resistance]; T3 --> P3((P3)); P3 --> T4[T4: recommend_bundle_with_justification]; T4 --> P4((P4)); P4 --> T5[T5: draft_alert_text_for_care_team]; T5 --> P5((P5));
```

telas/32-bancada-passo.png

BioByte Senti... LangNet

Buscar...

Voltar ao Dashboard

ACOMPANHAMENTO DE EXECUÇÃO - INPUTS/EXECUÇÃO/OUTPUTS

Projeto: Rede de Petri — Vigilância de ICSAC (BioByte) Data: 27/09/2026, 19:09:45 Servidor de agentes: ws://127.0.0.1:5025

TAREFAS DESTA REDE

1. translate_lab_result_to_hospital_vocabulary
2. classify_case_by_nhsn_criterion
3. evaluate_multidrug_resistance
4. recommend_bundle_with_justification
5. draft_alert_text_for_care_team

EXECUÇÃO EM TEMPO REAL

Iniciando execução via Simulador de Petri Net...

Simulador de Petri Net

Modo de Execução: Automático

Parar Simulação Reset

Simulação Ativa

Transições Aptas: Nenhuma transição apta

Estado Atual (Tokens):

- P0 (Início do Fluxo): 0 tokens
- P1 (Task translate_lab_result_to_hospital_vocabulary pronta): 0 tokens
- P2 (Task classify_case_by_nhsn_criterion pronta): 0 tokens
- P3 (Task evaluate_multidrug_resistance pronta): 0 tokens
- P4 (Task recommend_bundle_with_justification pronta): 0 tokens
- P5 (Task draft_alert_text_for_care_team pronta): 0 tokens

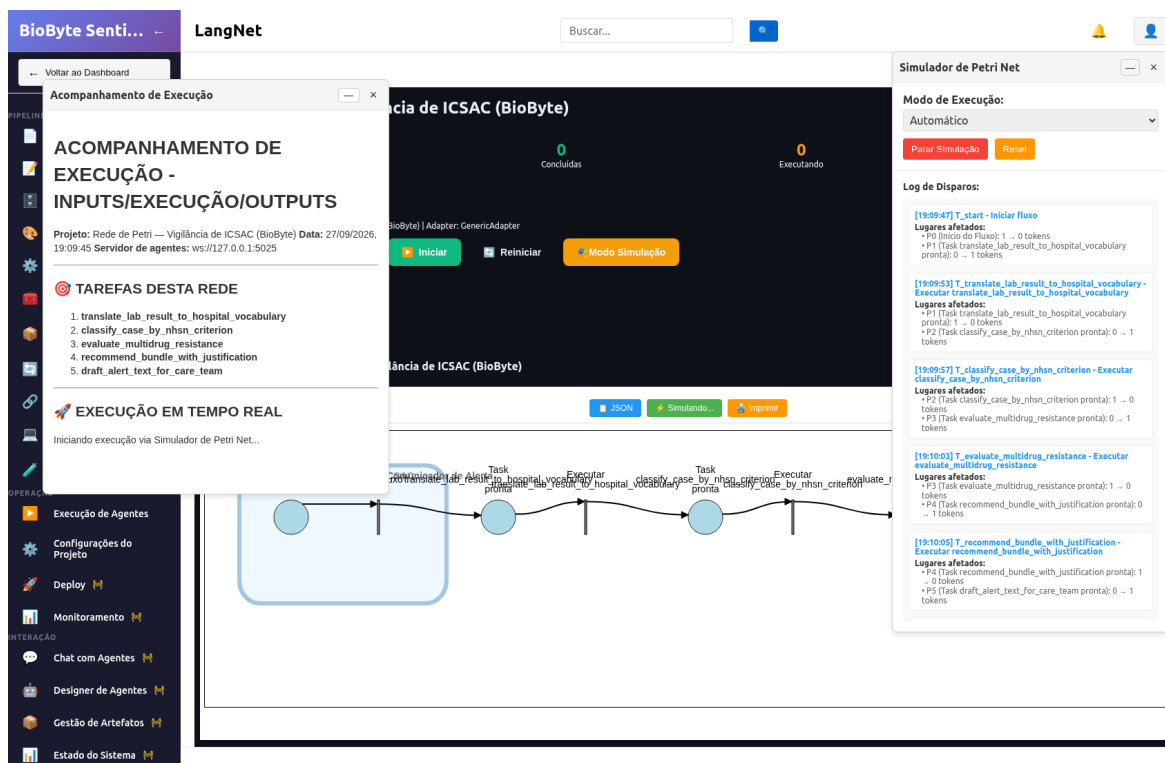
Log de Disparos:

- [19:09:47] T_start - Iniciar fluxo
Lugares afetados:
 - P0 (Início do Fluxo): 1 -> 0 tokens
 - P1 (Task translate_lab_result_to_hospital_vocabulary pronta): 0 -> 1 tokens
- [19:09:53] T_translate_lab_result_to_hospital_vocabulary - Executar translate_lab_result_to_hospital_vocabulary
Lugares afetados:
 - P1 (Task translate_lab_result_to_hospital_vocabulary pronta): 1 -> 0 tokens
 - P2 (Task classify_case_by_nhsn_criterion pronta): 0 -> 1 tokens
- [19:09:57] T_classify_case_by_nhsn_criterion - Executar classify_case_by_nhsn_criterion
Lugares afetados:
 - P2 (Task classify_case_by_nhsn_criterion pronta): 1 -> 0 tokens
 - P3 (Task evaluate_multidrug_resistance pronta): 0 -> 1 tokens

Diagrama de Petri Net:

```
graph LR; P0((P0)) --> T1[T1: translate_lab_result_to_hospital_vocabulary]; T1 --> P1((P1)); P1 --> T2[T2: classify_case_by_nhsn_criterion]; T2 --> P2((P2)); P2 --> T3[T3: evaluate_multidrug_resistance]; T3 --> P3((P3)); P3 --> T4[T4: recommend_bundle_with_justification]; T4 --> P4((P4)); P4 --> T5[T5: draft_alert_text_for_care_team]; T5 --> P5((P5));
```

telas/33-bancada-passo.png



telas/41-bancada-log-disparos.png

Bloco 4 — A prova

Cena 13 · A pergunta que não admite conversa

Telas: telas/32-bancada-passo.png (o painel com as cinco tarefas e o servidor no ar) e telas/41-bancada-log-disparos.png (o log de disparos)

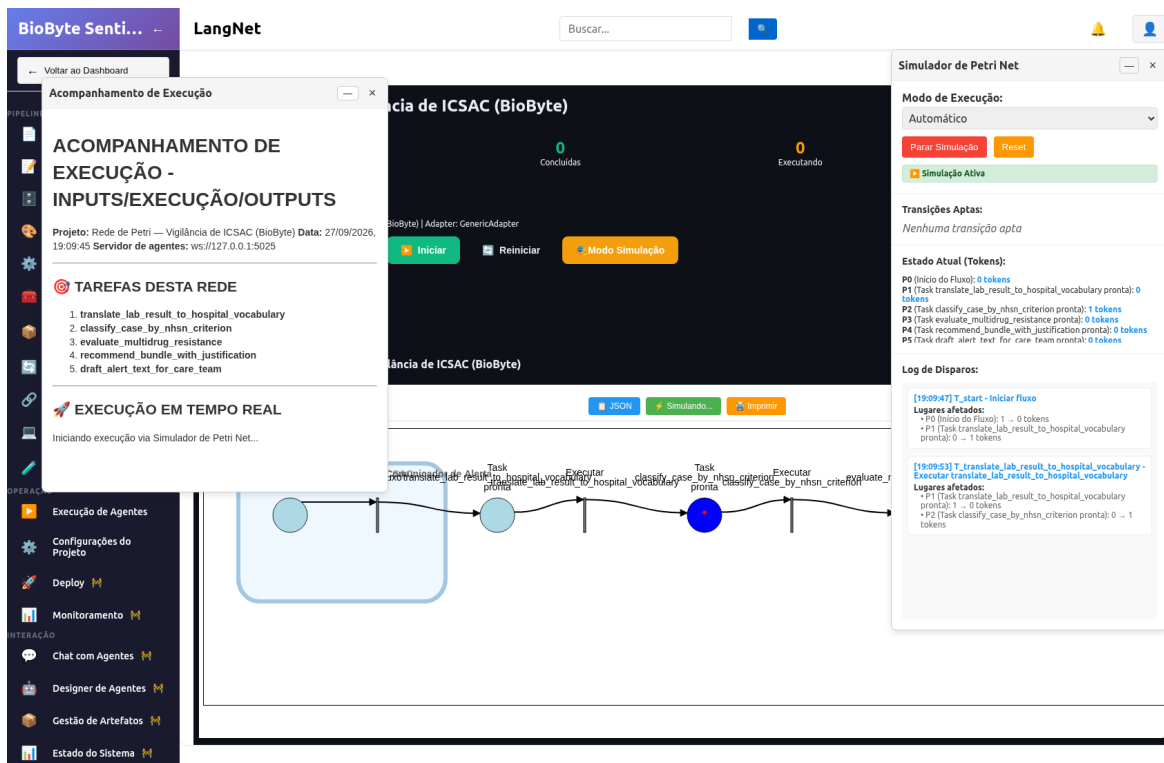
Narração:

Aqui está a prova. Plantamos no banco dois casos. No primeiro, o germe é resistente a quatro classes de antimicrobianos — pela regra, é multirresistente. No segundo, é resistente a uma só — não é. Se o sistema responder a mesma coisa nos dois, ele está inventando.

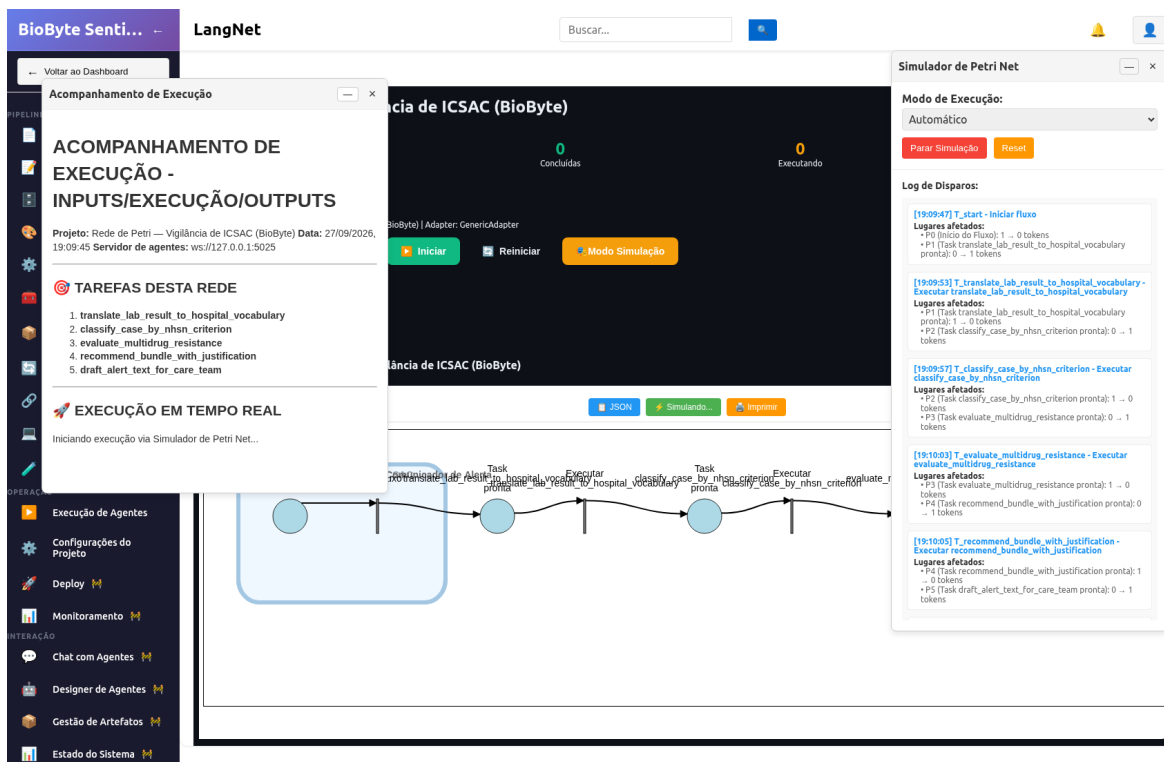
>

A tarefa consultou o banco, fez a junção entre antibiograma e antimicrobianos, e respondeu: no primeiro caso, quatro classes — cefalosporina, betalactâmico, fluoroquinolona e aminoglicosídeo. No segundo, uma: fluoroquinolona. Acertou os dois.

Produção: 28 segundos. Destaque os dois números lado a lado — quatro e um. Esta é a cena que sustenta o vídeo inteiro.



telas/32-bancada-passo.png



telas/41-bancada-log-disparos.png

Cena 13b · A bateria, contra o gabarito

Tela: telas/64-grafo-causa-efeito.png (retomada) — narração sobre o resultado da bateria

Narração:

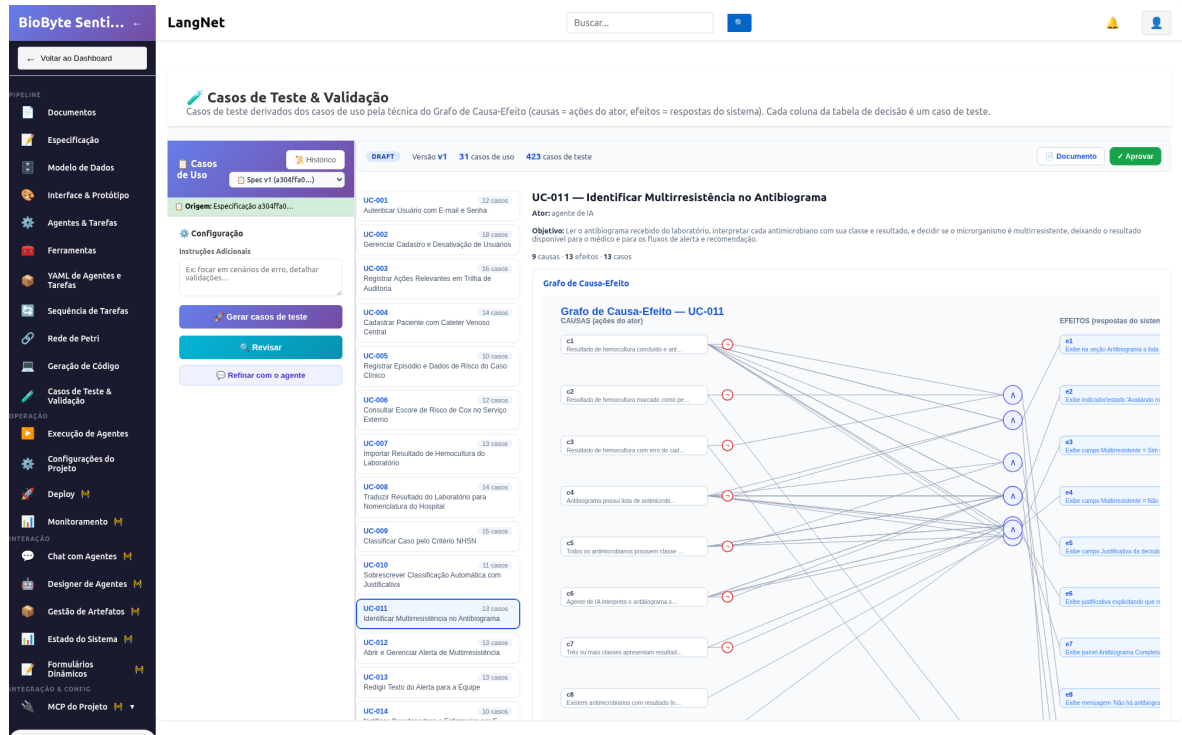
Os casos de teste não servem de enfeite. Plantamos no banco dois casos com gabarito conhecido e rodamos as tarefas do sistema contra eles. Sete dos nove casos passam, e passam três vezes

seguidas — não é sorte.

Passa a contagem das classes resistentes: quatro num caso, uma no outro. Passa a tradução do resultado do laboratório, com os seis antimicrobianos e as suas classes. Passa a recusa de uma amostra que não existe, sem inventar nada. Passa a recomendação do pacote, com a justificativa que o médico vai ler. E passa o texto do alerta.

Falham dois, e são o mesmo: a regra das três classes. O sistema conta certo e não conclui.

Produção: 30 segundos. Mostre o resultado da bateria em texto sobre a tela. Destaque o "7 de 9" e, logo depois, o "falham dois, e são o mesmo".



telas/64-grafo-causa-efeito.png

Cena 14 · O que ainda falta, dito na cara

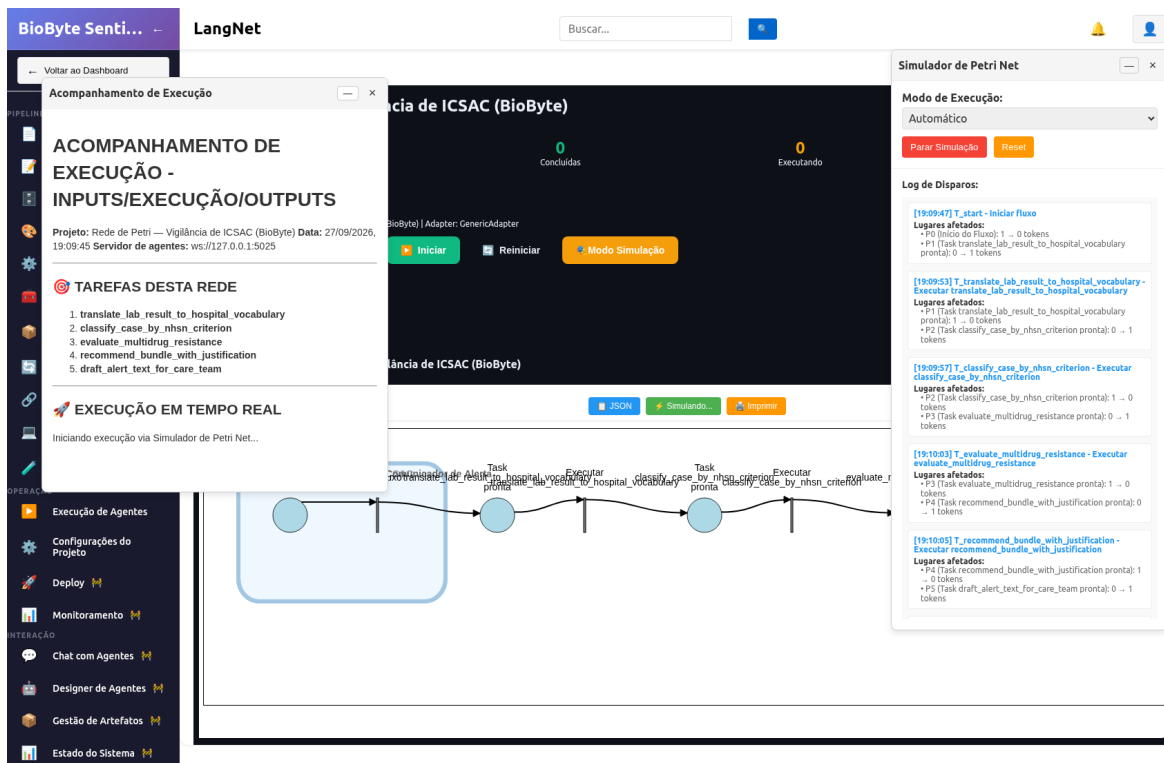
Tela: telas/40-bancada-fim.png

Narração:

E aqui está o que ainda não funciona. A conta saiu certa: quatro classes. Mas o campo que diz "é multirresistente, sim ou não" voltou vazio. A regra prática — resistência a três ou mais classes — não virou código, e o modelo também não a aplicou.

O interessante é que o sistema sabia disso antes de rodar. O portão de lógica, que confere se cada passo descrito na tarefa virou código, tinha apontado esse passo exatamente. Ele não impediu o defeito, mas o nomeou antes que alguém fosse enganado por ele.

Produção: 25 segundos. Destaque multirresistente: null ao lado de contagem_classes_resistentes: 4. Em seguida, o item correspondente na lista do portão.



telas/40-bancada-fim.png

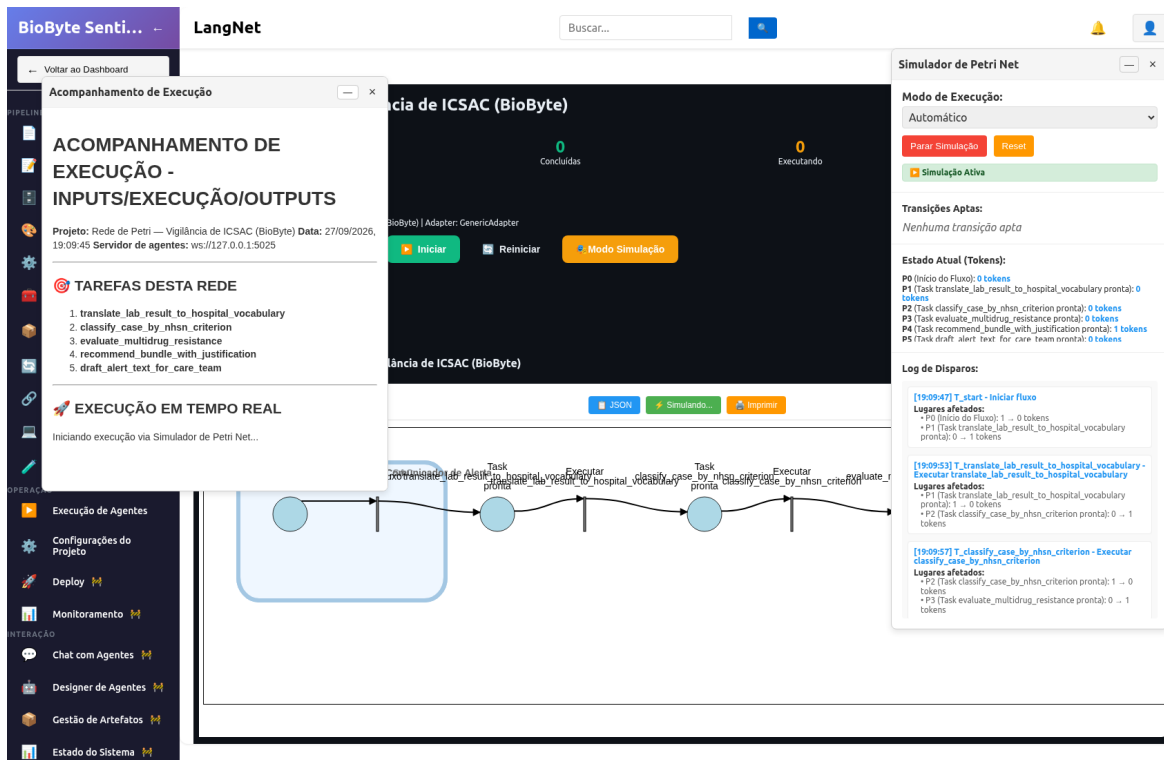
Cena 15 · Falha não passa calada

Tela: telas/34-bancada-passo.png

Narração:

E quando uma tarefa falha, a rede para. Esta tarefa não recebeu o que precisava e devolveu erro; a transição seguinte não disparou. É assim que tem de ser: um sistema que segue em frente com dado ruim entrega um laudo errado com cara de laudo certo.

Produção: 16 segundos. Destaque o lugar em vermelho e a transição que não disparou.



- A separação entre agêntico e convencional percorre requisitos, casos de uso, agentes e tarefas, interface e o menu do sistema gerado.
- Cinco tarefas em dois agentes, contra quarenta e cinco da versão anterior.
- As três ferramentas têm implementação declarada; a busca por semelhança acerta as três consultas de prova em primeiro lugar.
- Trinta e um casos de uso viram quatrocentos e vinte e três casos de teste, pelo grafo de causa e efeito.

- O protótipo navega as trinta telas com dados de exemplo.
- A rede roda inteira na Bancada: a marca passa pelas cinco tarefas e chega ao fim.
- **A bateria passa em 7 dos 9 casos, três rodadas seguidas:** a contagem das classes resistentes (4 e 1), a tradução completa do laboratório (6 antimicrobianos com classe), a recusa de amostra inexistente sem inventar, a classificação pela norma, a recomendação do pacote com justificativa e o texto do alerta.
- Quando uma tarefa falha, a transição seguinte não dispara.

Não provado — não narre como pronto:

- **A regra das três classes não é aplicada:** a contagem sai certa e o veredito volta vazio. São os 2 casos que a bateria reprova.
- As telas convencionais não têm executor: os botões não estão ligados a nada. A mesma trilha de auditoria que o protótipo navega aparece vazia no sistema gerado.
- Das cinco tarefas de agente, só duas são alcançáveis pelas telas do hospital; as outras três só rodam pela Bancada.
- A etapa de casos de teste aponta 36 casos contraditórios, onde falta na tabela a causa que dispara a exceção.
- A conferência de qualidade das tarefas aponta quatro elementos faltando em todas as cinco.